C# .NET 8.0 - OBJECT-ORIENTED PROGRAMMING

Student Grades Manager

DEPI

Digital Egypt Pioneers Initiative - Final Project Presentation

PRESENTED BY TEAM MEMBERS

S	Y	A	A	T
Salah	**Yousab**	**Alshiamaa**	**Azza**	**Tamer**
Project Contributor	Project Contributor	Project Contributor	Project Contributor	Project Contributor

Clean, modular console system for recording, calculating, and evaluating student grades.

OVERVIEW

Project Overview & Core Purpose

What is Student Grades Manager and how is it engineered?

CORE PURPOSE

Robust C# Console System for Academic Grade Management

A clean, modular object-oriented console application in .NET 8.0.

Automates student records, statistical computations, and performance reports with zero crashes.

Stack: C# .NET 8.0 | Strong OOP | Defensive Validation | In-Memory Storage

Domain Modeling

- Strongly-typed Student entity
- Immutable IReadOnlyList<int>
- Array.Copy defensive clone

Statistical Precision

- CalculateAverage() double precision
- GetHighestGrade() linear scan
- GetLowestGrade() linear scan

Defensive Stability

- int.TryParse user input guards
- Range bounds (Age 5-60, Marks 0-100)
- Enforced unique student IDs

PROBLEM VS SOLUTION

The Problem & How Our Code Solves It

Addressing traditional software vulnerabilities through defensive C# code.

THE PROBLEM

Vulnerabilities & Pitfalls

- **Calculation Bugs**
  Manual or integer division truncating decimal accuracy.
- **Dirty Input Values**
  Entering negative marks or scores above 100 corrupts data.
- **Duplicate Student IDs**
  Re-using the same ID causes data collisions and overwrites.
- **External Mutation**
  Exposing raw arrays allows outside code to tamper with grades.

THE SOLUTION

Our Code Implementation

- **GradeCalculator**
  (double)sum / count ensures exact float arithmetic.
- **InputValidator**
  Strict validation checks: IsValidGrade() and IsValidAge().
- **IsUniqueId Guard**
  Validates uniqueness against List<Student> before inserting.
- **Immutable Encapsulation**
  Array.Copy + Array.AsReadOnly wraps grades safely.

FEATURES

Key System Capabilities & Menus

Four primary application features accessible from the interactive console loop.

OPTION 1

Add Student

- Interactive CLI data collection
- Validates ID, Name, Age (5-60)
- Collects 1 to 7 grade marks (0-100)
- Creates immutable Student object

OPTION 2

Display All Students

- Prints formatted roster table
- Shows ID, Name, Age & Grades
- Uses string.Join(', ', student.Grades)
- Graceful empty message if no records

OPTION 3

Search by ID

- Fast numeric lookup via SearchStudentById()
- Returns nullable reference: Student?
- Displays full details when found
- Handles non-existent IDs cleanly

OPTION 4

Academic Report

- Calculates student average (F2 format)
- Displays highest & lowest marks
- Evaluates Pass / Fail standing
- Uses StudentReport.GetStudentStatus()

ARCHITECTURE

System Architecture & The 6 Project Files

Decoupled Single Responsibility Principle (SRP) codebase.

Program.cs

Entry & UI Loop

```
static void Main(string[])
```

Single Responsibility

Student.cs

Domain Entity

```
IReadOnlyList<int> Grades
```

Single Responsibility

StudentManager.cs

Data Repository

```
List<Student> students
```

Single Responsibility

GradeCalculator.cs

Math Engine

```
CalculateAverage(grades)
```

Single Responsibility

InputValidator.cs

Validation Layer

```
IsValidGrade(int grade)
```

Single Responsibility

StudentReport.cs

Status Evaluator

```
GetStudentStatus(avg)
```

Single Responsibility

CLASSES DEEP DIVE

Student.cs & StudentManager.cs

Data encapsulation, constructor guard clauses, and in-memory management.

DOMAIN MODEL

Student.cs

```
public class Student
```

```
public int ID { get; }
```

```
public string Name { get; }
```

```
public int Age { get; }  // (5 - 60)
```

```
public IReadOnlyList<int> Grades { get; }
```

Key Defensive Highlights:

- Constructor Guard Clauses for all properties
- Array.Copy(grades, gradesCopy, grades.Length)
- Array.AsReadOnly(gradesCopy) prevents tampering
- Enforces 1 to 7 grades, each bounded to 0-100

REPOSITORY

StudentManager.cs

```
public class StudentManager
```

```
private readonly List<Student> students;
```

```
public void AddStudent()
```

```
public void DisplayStudents()
```

```
public Student? SearchStudentById(int id)
```

Key Implementation Highlights:

- Private list encapsulation prevents direct external clearing
- AddStudent() uses int.TryParse & validator checks
- DisplayStudentDetails() centralizes formatting
- Nullable Student? return type for clean search

CLASSES DEEP DIVE

GradeCalculator.cs & StudentReport.cs

Statistical accuracy, shared validation helper, and clear pass/fail status.

MATH ENGINE

GradeCalculator.cs

```
public class GradeCalculator
```

```
private void ValidateGrades(IReadOnlyList<int>)
```

```
public double CalculateAverage(grades)
```

```
public int GetHighestGrade(grades)
```

```
public int GetLowestGrade(grades)
```

Code Highlights:

- ValidateGrades(): Reusable check for null, empty, 0-100
- CalculateAverage(): (double)sum / grades.Count
- GetHighestGrade(): Linear scan comparing against maxGrade
- GetLowestGrade(): Linear scan comparing against minGrade

EVALUATION RULE

StudentReport.cs

```
public class StudentReport
```

```
public string GetStudentStatus(double avg)
```

```
return average >= 50 ? "Pass" : "Fail";
```

Architectural Purpose:

- Single Responsibility: Exclusively evaluates standing
- Concise Ternary Expression: No bloated if/else logic
- Universal 50.0% standard passing threshold
- Decoupled from math & UI: Pure business rule

CLASSES DEEP DIVE

InputValidator.cs & Program.cs

Boolean validation rules and console runtime orchestration.

VALIDATION LAYER

InputValidator.cs

```
public class InputValidator
```

```
IsValidId(id) => id > 0
```

```
IsValidName(name) => !IsNullOrWhiteSpace
```

```
IsValidAge(age) => age >= 5 && age <= 60
```

```
IsValidGrade(g) => g >= 0 && g <= 100
```

```
IsUniqueId(id, list) => !exists in list
```

Why Clean Booleans?

- Eliminates duplicate validation logic across app
- Provides clear true/false contracts
- Throws ArgumentNullException if student list is null

UI DISPATCHER

Program.cs

```
internal class Program
```

```
static void Main(string[] args)
```

```
while (running) { switch(choice) }
```

```
SearchStudent(manager)
```

```
GenerateStudentReport(mgr, calc, rep)
```

Console Runtime Design:

- Interactive menu loop with options 1 to 5
- Modular helper methods keep Main concise
- Safe numeric parsing via int.TryParse
- Zero math or storage logic: pure UI dispatcher

SOFTWARE ENGINEERING

OOP Principles Applied in Code

Direct correlation between theoretical design patterns and our C# codebase.

PRINCIPLE 01

Encapsulation & Immutability

- Get-only properties { get; }
- Array.Copy() defensive copying
- IReadOnlyList<int> grade protection

PRINCIPLE 02

Single Responsibility (SRP)

- GradeCalculator calculates only
- InputValidator validates only
- StudentReport evaluates standing only

PRINCIPLE 03

Separation of Concerns (SoC)

- Program.cs manages only Console UI
- Domain entities isolated from UI
- Engine classes ready for Web/GUI reuse

PRINCIPLE 04

Defensive Programming

- int.TryParse prevents format crashes
- ?? string.Empty eliminates null warnings
- Strict range checks on age and marks

CONCLUSION

Summary & Thank You

Key achievements, future roadmap, and DEPI team recognition.

DELIVERED

Project Highlights

- **Clean C# OOP Architecture**
  Structured across 6 decoupled files.
- **Defensive Input Validation**
  Zero unhandled exceptions or corrupt data.
- **Accurate Grade Analytics**
  Automated average, highest, lowest & status.
- **Protected Immutability**
  State is guarded against external mutations.

FUTURE WORK

Future Enhancements

- **Database Persistence**
  Integrate SQLite or Entity Framework Core.
- **Modern UI**
  Build a desktop GUI (WPF) or Web App (Blazor).
- **Export Reports**
  Direct export to Excel and PDF formats.
- **GPA & Credit Hours**
  Support course weighting and 4.0 GPA scale.

Thank You!

Digital Egypt Pioneers Initiative (DEPI) - Project Presented by:

Salah **Yousab** **Alshiamaa** **Azza** **Tamer**

Questions & Discussion Welcome